VITyarthi Project Report

SMART CAMPUS RESOURCE OPTIMIZATION SYSTEM

A Python-based modular system for campus resource registration, usage analysis, inefficiency detection and optimization recommendations.

Student Name	Shreya Koushal
Registration Number	26MIM10148
Course	VITyarthi – Build Your Own Project
Technology	Python

Prepared according to the supplied VITyarthi project-report guidelines.

1. Introduction

The Smart Campus Resource Optimization System is a console-based Python project designed to organize and analyze information related to campus resources. It provides a menu-driven interface for resource registration, usage calculation, utilization analysis, inefficiency detection, optimization recommendations, report generation, and data validation.

The main program imports seven module functions and provides menu options 1–7 for the project functions and option 8 for exit. ■filecite■turn0file0■L3-L9■ ■filecite■turn0file0■L15-L21■

The supplied VITyarthi guidelines require students to identify a meaningful problem, design a technical solution, implement it using course concepts, and document and evaluate the solution. ■filecite■turn0file8■L14-L20■

2. Problem Statement

Campus resources such as electricity, water, classrooms and laboratories can experience changing levels of usage. A simple structured system can help record resource details, calculate changes in consumption, classify utilization, identify possible inefficiency and provide basic recommendations. This project implements those operations through a modular Python application.

3. Objectives

- Provide a simple menu-driven interface for campus resource analysis.
- Register resource ID, name, type, location and capacity.
- Calculate usage difference and percentage change.
- Calculate utilization percentage against capacity.
- Detect usage increases and possible inefficiency.
- Generate resource-specific optimization recommendations.
- Generate a concise analysis report and validate resource data.

4. Functional Requirements

The guidelines require at least three major functional modules, clear input/output structure and a logical workflow. ■filecite■turn0file8■L24-L37■ The submitted implementation provides seven modules:

- Resource Registration – collects resource details and stores resource dictionaries in a list. ■filecite■turn0file1■L5-L21■
- Resource Usage Calculation – calculates usage difference and percentage change. ■filecite■turn0file2■L7-L20■
- Resource Utilization Analysis – calculates utilization and classifies it as underutilized, efficiently utilized or highly utilized. ■filecite■turn0file3■L10-L17■ ■filecite■turn0file3■L27-L34■
- Inefficiency and Wastage Detection – identifies high, moderate, normal or reduced usage and displays an alert. ■filecite■turn0file4■L17-L30■ ■filecite■turn0file4■L33-L43■
- Optimization and Recommendation – gives resource-specific recommendations for electricity, water, classrooms and laboratories. ■filecite■turn0file5■L27-L50■
- Report Generation – creates a resource analysis report with status and recommendation. ■filecite■turn0file6■L23-L54■
- Validation and System Control – validates numeric capacity and important text fields. ■filecite■turn0file7■L14-L25■ ■filecite■turn0file7■L27-L44■

5. Non-Functional Requirements

The guidelines require at least four non-functional requirements and list areas such as performance, usability, reliability, maintainability, error handling and resource efficiency. ■filecite■turn0file8■L38-L50■

- Usability: numbered menu options and direct input prompts.
- Maintainability: separate files/functions for separate operations.
- Reliability: validation and conditional checks for invalid values.
- Error handling: Module 7 catches ValueError for non-numeric capacity. ■filecite■turn0file7■L14-L25■
- Resource efficiency: simple arithmetic and control structures are used.
- Scalability: additional modules can be connected through the central menu.

6. System Architecture

The application uses a modular controller architecture. The main program imports the seven module functions and calls the selected function according to the user's menu choice. ■filecite■turn0file0■L3-L9■

System Architecture Diagram

USER
↓
MAIN PROGRAM / MENU CONTROLLER
↓
1 Registration | 2 Usage Calculation | 3 Utilization Analysis
4 Inefficiency Detection | 5 Optimization | 6 Report | 7 Validation
↓
RESULT / STATUS / RECOMMENDATION

7. Design Diagrams

7.1 Use Case Diagram

User → Register Resource
User → Calculate Usage
User → Analyze Utilization
User → Detect Inefficiency
User → Get Recommendation
User → Generate Report
User → Validate Data
User → Exit

7.2 Workflow Diagram

Start → Display Menu → Enter Choice → Execute Selected Module → Display Result → Continue? → Yes: Menu / No: Exit → End

7.3 Sequence Diagram

User → Main Program: select option
Main Program → Module: call function
Module → User: request data
User → Module: enter data
Module → Main Program: finish/return
Main Program → User: result / next menu

7.4 Class/Component Diagram

Main Program
■■■ Module 1: Resource Registration
■■■ Module 2: Usage Calculation
■■■ Module 3: Utilization Analysis

■■■ Module 4: Inefficiency Detection
■■■ Module 5: Optimization Recommendation
■■■ Module 6: Report Generation
■■■ Module 7: Validation & System Control

7.5 ER / Storage Design

The supplied code does not use a database. Module 1 uses a Python list containing resource dictionaries with ID, Name, Type, Location and Capacity. ■filecite■turn0file1■L15-L21■ Therefore, a persistent ER diagram is not applicable to the current implementation.

8. Design Decisions & Rationale

- Modular programming separates each major operation into its own function/file.
- The central menu provides a clear user workflow.
- Lists and dictionaries are used for temporary resource information.
- Arithmetic formulas perform usage and utilization calculations.
- Conditional statements classify resource status and select recommendations.
- try/except and empty-field checks provide basic input validation.

9. Implementation Details

9.1 Main Program

The main program imports the seven functions and maps menu choices 1–7 to the respective modules. Choice 8 exits, while invalid choices produce an error message. ■filecite■turn0file0■L26-L57■

9.2 Module Summary

Module	Implementation
1	Resource registration using a list, dictionaries and a repetition loop.
2	Usage difference and percentage change using arithmetic and conditional logic.
3	Capacity validation, utilization percentage and utilization classification.
4	Percentage increase and inefficiency/wastage classification.
5	Usage-change classification and resource-type-specific recommendations.
6	Resource analysis report with status and recommendation.
7	Capacity validation with try/except and empty-field checks.

9.3 Key Formulas

- Usage Difference = Current Usage − Previous Usage.
- Percentage Change = (Usage Difference / Previous Usage) × 100 when previous usage is not zero. ■filecite■turn0file2■L12-L20■
- Utilization Percentage = (Actual Usage / Capacity) × 100. ■filecite■turn0file3■L10-L17■

10. Screenshots / Results

No screenshots were supplied with the project files, so this report does not fabricate screenshots. The following result layouts are derived directly from the submitted program's output structure.

Main Menu

```
SMART CAMPUS MANAGEMENT SYSTEM

1. Resorce Registration
2. Resource Usage Calculation
3. Resource Utilization Analysis
4. Inefficiency and Wastage Detection
5. Optimization and Recommendation
6. Report Generation
7. Validation and System Control
8. Exit
```

Resource Analysis Report

```
RESOURCE ANALYSIS REPORT
Resource Id
Resource Name
Resource Type
Location

RESOURCE STATUS
Status

RECOMMENDATION
Recommendation text
```

11. Testing Approach

The VITyarthi guidelines require testing wherever applicable, including unit tests or validation tests. ■filecite■turn0file8■L60-L63■ The following test cases correspond to branches present in the supplied code.

Area	Condition	Expected behavior
Menu	Choice 1–7	Corresponding module executes.
Menu	Choice 8	System exits.
Menu	Invalid choice	Invalid-choice message.
Usage	Previous usage ≠ 0	Percentage change calculated.
Usage	Previous usage = 0	Percentage change set to 0.
Utilization	Capacity ≤ 0	Invalid-capacity message.
Inefficiency	>30% increase	High inefficiency/wastage alert.
Optimization	Electricity/water/classroom/laboratory	Relevant recommendation branch.
Validation	Non-numeric capacity	ValueError handled.
Validation	Empty ID/name/type/location	Invalid-data message.

12. Challenges Faced

- Integrating seven separate modules through one main menu.
- Handling different resource types for tailored recommendations.
- Applying validation to numeric and text inputs.
- Defining usage thresholds for status messages.
- Keeping the project within a core-Python console implementation.

13. Learnings & Key Takeaways

- Modular programming with functions and imports.
- Lists and dictionaries for structured temporary data.
- Loops for repeated user interaction.
- Conditional statements for classification and decision-making.
- Arithmetic formulas for percentage calculations.
- try/except for input error handling.
- Designing a central controller for multiple modules.

14. Future Enhancements

- Add persistent database or file storage.
- Connect registered resources directly to later analysis modules.
- Add graphical dashboards and trend charts.
- Add authentication and role-based access.
- Store historical resource usage.
- Add automatic alerts for threshold violations.
- Add automated tests and Git-based version control.

15. References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines. ■filecite■turn0file8■L3-L20■
- Submitted Main Program and Modules 1–7, used as the implementation basis for this report. ■filecite■turn0file0■L3-L9■
- VITyarthi Project Report Submission requirements. ■filecite■turn0file8■L109-L134■

Appendix A – Submitted Source Code

The following appendix reproduces the supplied Python files as the implementation reference used in preparing this report.

Main Program

```
def main_function():

    from MODULE1 import resource_registration
    from MODULE2 import resource_usage_calculation
    from MODULE3 import resource_utilization_analysis
    from MODULE4 import inefficiency_detection
    from MODULE5 import optimization_recommendation
    from MODULE6 import report_generation
    from MODULE7 import validate_resource_data

    print("\n**************************************************************************")
    print("                     SMART CAMPUS MANAGEMENT SYSTEM                      ")
    print("\n**************************************************************************")

    print("\n1. Resorce Registration")
    print("2. Resource Usage Calculation")
    print("3. Resource Utilization Analysis")
    print("4. Inefficiency and Wastage Detection")
    print("5. Optimization and Recommendation")
    print("6. Report Generation")
    print("7. Validation and System Control")
    print("8. Exit")

    y='yes'

    while y=='yes':
        choice=input("\nEnter your choice :")

        if choice=='1':
            resource_registration()

        elif choice=='2':
            resource_usage_calculation()

        elif choice=='3':
            resource_utilization_analysis()

        elif choice=='4':
            inefficiency_detection()

        elif choice=='5':
            optimization_recommendation()

        elif choice=='6':
            report_generation()

        elif choice=='7':
            validate_resource_data()

        elif choice=='8':
            print("\nExiting the system...!")
            break

        else:
            print("\nInvalid choice! Please enter the number from 1 to 8.")
            n=input('Enter yes for continue :')
            y=n.lower()

main_function()
```

Module 1

```
# Module 1 : Resource Registration

def resource_registration():

    resources=[]
    print("\n*~~~~~~~~~~* RESOURCE REGISTRATION *~~~~~~~~~~*")

    while True:
        resource_id=input("Enter the resource ID :")
        resource_name=input("Enter the resource name :")
        resource_type=input("Enter the resource type :")
        location=input("Enter the location :")
        capacity=input("Enter the capacity :")

        resource={"ID": resource_id,
            "Name": resource_name,
            "Type": resource_type,
            "Location": location,
            "Capacity": capacity}

        resources.append(resource)

        print("\nResource Registered Successfully!!!")

        choice = input("\nDo you want to register another resource? (yes\no) :")

        if choice.lower()!="yes":
            break

    print("\n*~~~~~~~~~~* REGISTERED RESOURCES *~~~~~~~~~~*")

    for resource in resources:
        print("\nResource Id     :-    ",resource_id)
        print("Resource Name     :-    ",resource_name)
        print("Resource Type     :-    ",resource_type)
        print("Location          :-    ",location)
        print("Capacity          :-    ",capacity)

    return resources
```

Module 2

```
# Module 2 : Resource Usage calculation

def resource_usage_calculation():

    print("\n*~~~~~~~~~~* RESOURCE USAGE CALCULATION *~~~~~~~~~~*")

    resource_id=input("Enter the resource ID :")
    resource_name=input("Enter the resource name :")

    print("\nEnter resource usage details ")

    previous_usage=float(input("Enter previous usage :"))
    current_usage=float(input("Enter current usage :"))

    usage_difference=current_usage-previous_usage

    if previous_usage!=0:
        percentage_change=(usage_difference/previous_usage)*100
    else:
        percentage_change=0


    print("\n*~~~~~~~~~~* USAGE RESULT *~~~~~~~~~~*")

    print("\nResource Id     :-    ",resource_id)
    print("Resource Name     :-    ",resource_name)
    print("Previous Usage    :-    ",previous_usage)
    print("Current Usage     :-    ",current_usage)
    print("usage Difference  :-    ",usage_difference)
    print("Percentage Change :-    ",round(percentage_change,2),'%')
```

Module 3

```
# Module 3 : Resource Utilization Analysis

def resource_utilization_analysis():

    print("\n*~~~~~~~~~~* RESOURCE USAGE CALCULATION *~~~~~~~~~~*")

    resource_id=input("Enter the resource ID :")
    resource_name=input("Enter the resource name :")

    capacity=float(input("Enter total capacity :"))
    actual_usage=float(input("Enter actual usage :"))

    if capacity<=0:
        print("\nInvalid capacity. Capacity must be greater than 0")
        return

    utilization_percentage=(actual_usage/capacity)*100

    print("\n*~~~~~~~~~~* UTILIZATION RESULT *~~~~~~~~~~*")

    print("\nResource Id     :-    ",resource_id)
    print("Resource Name     :-    ",resource_name)
    print("Total Capacity    :-    ",capacity)
    print("Actual usage      :-    ",actual_usage)
    print("Utilization       :-    ",round(utilization_percentage,2),'%')

    if utilization_percentage<50:
        status="Underutilized"

    elif utilization_percentage<=80:
        status="Efficiently Utilized"

    else:
        status="Highly Utilized"

    print("Utilization Status :",status)
```

Module 4

```
# Module 4 : Inefficiency and Waste Management

def inefficiency_detection():

    print("\n*~~~~~~~~~~* RESOURCE USAGE CALCULATION *~~~~~~~~~~*")

    resource_id=input("Enter the resource ID :")
    resource_name=input("Enter the resource name :")

    previous_usage=float(input("Enter previous usage :"))
    current_usage=float(input("Enter current usage :"))

    if previous_usage<=0:
        print("\nInvalif previous usage.")
        return

    usage_difference=current_usage-previous_usage
    percentage_increase=(usage_difference/previous_usage)*100

    print("\n*~~~~~~~~~~* ANANLYSIS RESULT *~~~~~~~~~~*")

    print("\nResource Id     :-    ",resource_id)
    print("Resource Name     :-    ",resource_name)
    print("Previous Usage    :-    ",previous_usage)
    print("Current Usage     :-    ",current_usage)
    print("Usage Increase    :-    ",round(usage_difference,2))
    print("Percentage Increase :-   ",round(percentage_increase,2),'%')

    if percentage_increase>30:
        print("Status    :    High Inefficiency Detected")
        print("Alert     :    Possible resource wastage.")

    elif percentage_increase>10:
        print("Status    :    Moderate Increase")
        print("Alert     :    Resource usage should be monitored.")

    elif percentage_increase>=0:
        print("Status    :    Normal Usage")
        print("Alert     :    No significant inefficiency detected.")

    else:
        print("Status    :    Reduced Usage")
        print("Alert     :    Resource consumption has decreased.")
```

Module 5

```
# Module 5 : Optimization and Recommendation

def optimization_recommendation():

    print("\n*~~~~~~~~~~* RESOURCE USAGE CALCULATION *~~~~~~~~~~*")

    resource_id=input("Enter the resource ID :")
    resource_name=input("Enter the resource name :")
    resource_type=input("Enter the resource type :")

    previous_usage=float(input("Enter previous usage :"))
    current_usage=float(input("Enter current usage :"))

    if previous_usage<=0:
        print("\nInvalid previous usage.")
        return

    percentage_change=((current_usage-previous_usage)/previous_usage)*100

    print("\n*~~~~~~~~~~* OPTIMIZATION ANALYSIS *~~~~~~~~~~*")

    print("\nResource Id     :-    ",resource_id)
    print("Resource Name     :-    ",resource_name)
    print("Resource Type     :-    ",resource_type)
    print("Usage Change      :-    ",round(percentage_change,2),'%')

    print("\nRecommendation :")

    if percentage_change>30:
        print("High resource consumption detected :-")

        if resource_type.lower()=="electricity":
            print("Switch off unecessary electrical equipment.")
            print("Monitor equipment during during low-occupancy periods.")
            print("Review operating hours.")

        elif resource_type.lower()=="water":
            print("Check for possible water leakage.")
            print("Reduce unnecessary water usage.")
            print("Monitor water consumption regularly.")

        elif resource_type.lower()=="classroom":
            print("Review classroom utilization.")
            print("Consider reallocating classes according to occupancy.")
        elif resource_type.lower()=="laboratory":
            print("Review laboratory usage hours.")
            print("Avoid unnecessary operation of laboratory equipment.")

        else:
            print("Monitor the resource regularly.")
            print("Investigate the reason for increased consumption.")
            print("optimize usage during low-demand periods.")

    elif percentage_change>10:
        print("Moderate increase in resource usage detected.")
        print("Continue monitoring the resource.")
        print("Compare future usage with the average.")
        print("Identify possible cause of increased consumption.")

    elif percentage_change>=0:
        print("Resource usage is within the normal range.")
        print("Continue regular monitoring.")
        print("Maintain efficient resource usage.")

    else:
        print("Resource usage has decreased.")
        print("Current usage is improving.")
        print("Maintain efficient resource usage.")
```

Module 6

```
# Module 6 : Report Generation

def report_generation():

    print("\n*~~~~~~~~~~* RESOURCE REGISTRATION *~~~~~~~~~~*")

    resource_id=input("Enter the resource ID :")
    resource_name=input("Enter the resource name :")
    resource_type=input("Enter the resource type :")
    location=input("Enter the location :")

    previous_usage=float(input("Enter previous usage :"))
    current_usage=float(input("Enter current usage :"))

    if previous_usage<=0:
        print("\nInvalid previous usage.")
        return

    usage_difference=current_usage-previous_usage

    percentage_change=((usage_difference/previous_usage)*100)

    print("\n*~~~~~~~~~~* RESOURCE ANALYSIS REPORT *~~~~~~~~~~*")

    print("\nResource Id     :-    ",resource_id)
    print("Resource Name     :-    ",resource_name)
    print("Resource Type     :-    ",resource_type)
    print("Loaction          :-    ",location)

    print("\n*~~~~~~~~~~* RESOURCE STATUS *~~~~~~~~~~*")

    if percentage_change>30:
        status="High Inefficiency"
        recommendation="Immediate optimization is recommended"

    elif percentage_change>10:
        status="Moderate increase"
        recommendation="Resource usage should be monitored"

    elif percentage_change>=0:
        status="Normal"
        recommendation="Continue efficient resource utilization"

    else:
        status="Improved"
        recommendation="Resource consumption has decreased"

    print("Status    :    ",status)

    print("\n*~~~~~~~~~~* RECOMMENDATION *~~~~~~~~~~*")

    print(recommendation)

    print("\n*~~~~~~~~~~* END OF REPORT *~~~~~~~~~~*")
```

Module 7

```
# Module 7 : Validation and System Control

def validate_resource_data():

    print("\n*~~~~~~~~~~* DATA VALIDATION AND SYSTEM CONTROL *~~~~~~~~~~*")

    resource_id=input("Enter the resource ID :")
    resource_name=input("Enter the resource name :")
    resource_type=input("Enter the resource type :")
    location=input("Enter the location :")

    print("\nEnter resource capacity ")

    try:
        capacity=float(input("Capacity :"))

        if capacity<=0:
            print("\nInvalid data!")
            print("Capacity must be greater than 0.")
            return

    except ValueError:
        print("\nInvalid Data!")
        print("Capacity must be number.")
        return

    if resource_id.strip()=="":
        print("\nInvalid Data!")
        print("Resource ID cannot be empty")
        return

    if resource_name.strip()=="":
        print("\nInvalid Data!")
        print("Resource name cannot be empty.")

    if resource_type.strip()=="":
        print("\nInvalid Data!")
        print("Resource type cannot be empty.")
        return

    if location.strip()=="":
        print("\nInvalid Data!")
        print("Location cannot be empty.")
        return

    print("\n*~~~~~~~~~~* VALIDATION RESULT *~~~~~~~~~~*")

    print("\nResource Id     :-    ",resource_id)
    print("Resource Name     :-    ",resource_name)
    print("Resource Type     :-    ",resource_type)
    print("Loaction          :-    ",location)
    print("Capacity          :-    ",capacity)

    print("\nData Validation Successful!!!")
    print("All entered information is valid.")
```